Volume 1 | Issue 1 | 2026

International Journal

of Multidisciplinary Science, Engineering & Technology. (IJMSET)

Comparative Analysis of AlexNet, GoogLeNet, VGG19, ResNet50, and ResNet101 for Improved Plant Disease Detection Through Convolutional Neural Networks

Harish Kumar

Department of Electrical and Electronics Engineering

School of Advanced Engineering, UPES, Dehradun 248007, India

Abstract

To meet the increasing need for food from a growing global population, it is imperative to make progress in Agri-cultural technology, namely in the area of disease detection. This will help reduce the need for pesticides and improve the overall health of crops. The issue at hand is the need for accurate and effective identification of plant diseases to enhance food production. This research tackles this problem by using cutting-edge CNNs on pre-existing agricultural datasets to assess and compare their effectiveness. Existing research does not show the precise and detailed performance of the latest CNN models in agricultural data sets. We addressed this issue by examining five CNN models: AlexNet, GoogLeNet, VGG19, ResNet50, and ResNet101. This approach involved meticulous training and assessment of these models to compare their precision, loss, duration of training, and complexity of the model. ResNet101 outperformed all other models with an accuracy rate of approximately 97%. Nevertheless, the practical implementation of this technology is hindered by its intricate model complexity and demanding resource needs, which make it unsuitable for low-power applications. GoogLeNet demonstrated superior performance in terms of accuracy, model complexity, training time, and other evaluation measures.

Keywords- Image processing, CNN, Dataset, Deep Learning, Recall, ROC.

1. INTRODUCTION

Agriculture is the most common occupation in Bangladesh. Around 70 % of the population depends on agriculture for their livelihood. Today, this sector is incorporating cuttingedge technologies to increase its production. In the realm of agricultural robotics, numerous research has been conducted on various projects. This paper builds upon the work presented in [1], which focused on an autonomous solar-powered robot system. Notably, the system utilized a Raspberry Pi for image processing. The study employed two distinct datasets, both of which were subsequently trained using two CNN architectures: VGG16 and a customized version of VGG16. Careful monitoring of the plant from an early age is foremost. It includes different stages such as soil preparation, seeding, spraying of water and pesticides, harvesting, and storage [2]. Pesticides are used to attract, seduce, and destroy harmful insects. Among the mentioned steps, pesticides need more carefulness [3]. Using the right amount and using it at the right time is one of the crucial tasks for farmers. Thus, accurate detection of plant diseases in the early stages can reduce pesticide use. This paper has tried to extend the work by comparing the latest CNN architecture accuracy. The VGG16 model has some limitations regarding the fact that it is slow to train. Also, it takes quite a large amount of disk space and bandwidth, which results in inefficiency. This model has issues like gradient problems due to a higher number of parameters. The average accuracy of the VGG16 (customized) model was 96.6 % on the previously trained dataset. Therefore, this paper used the latest CNN models in the data set and tried to analyze the result to find the most accurate model for the data set in

terms of training time, error rates, loss, computational resources, and other important parameters. Improved precision in the diagnosis of plant diseases will lead to a reduction in the use of pesticides. Robots will possess the ability to identify the disease in its initial phase and try to eradicate it by spraying pesticides. In this way, fewer pesticides will be needed, and the health of the plant will remain good. Hence, it will be economically beneficial to farmers.

2. RELATED WORK

Till now, various Image processing techniques have been used to detect plant diseases. In [4], CNN is used in an IOT-based robotic system for the detection of plant disease. A ConvNet is used to train 1000 images of four classes. As illustrated in Fig. 01, a CNN typically comprises a fully connected layer, a pooling layer, a convolutional layer, and a non-linearity layer.; thus, it is very efficient in learning new features from data sets. There have been two phases mentioned for disease classification. Firstly, input images were segmented and the patient portion was detected. In the second phase, features of the images were extracted and then classified the extracted features. This model was able to detect accurate classes with 96.73% of success rate.

The author of [5] used 15 classes of datasets from the internet. We started with the augmented process, which increased the data set and also reduced the complexity of overfitting. For neural network training, they used the CaffeNet framework, and It is a suitable framework for industrial deployment and research work. It is a deep CNN model that learns features from input images. Multiple layers consist of eight learning layers, five convolutional and three fully connected layers, making it easy to understand features from images. This model was able to detect an overall 96.3% accuracy.

Reference [6] used a Raspberry Pi as a controller as it has many functions, such as a computer. It was incorporated with different sensors such as soil moisture, temperature, and humidity and a USB port camera to monitor plants. OpenCV was installed in Raspberry Pi for image processing and computer vision. Image processing was used in the article to detect plant diseases. It included different OpenCV libraries, such as masking, segmentation, feature extraction, and so on. In the steps of processing the images, first captured images were saved in Raspberry, and then RGB image acquisition was done. The RGB images are then converted to HSV. Image boundaries were fixed, and the K-mean algorithm was applied to images. The green pixels were masked. Finally, by measuring the unmasked and masked pixels in the image, the diseased leaf was identified.

Reference [7] presents an overview of research works on plant disease detection. Diseased leaves were picked up from the plant. The pictures were taken carefully. The following steps were taken in the image processing segment: image enhancement, classification, and image segmentation. After that, image features were extracted. Most researchers extract plant disease features based on texture features. Plant diseases were classified into different types. Based on that, a diagnosis of the disease was made.

Reference [8] introduced a prototype that uses renewable energy and is unified by two functions- automated irrigation and plant disease detection. Here, in the deep learning model,

Volume 1 | Issue 1 | 2026
International Journal
of Multidisciplinary Science, Engineering & Technology. (IJMSET)

additional fully connected layers were tuned to gain better feature learning and class assignment. Several state-of-the-art architectures, such as Resnet50, AlexNet, and EfficientNet, were used, along with the tuned model. Among them, the tuned model had the highest accuracy of 99.24%.

3. METHODOLOGY

For training, this project had chosen the PlantVillage [9], as it contains a large number of datasets. This dataset includes 38 different classes of diseased images and healthy plants images, including 3172 images of apple, 1502 images of blueberry, 1906 images of cherry, 3852 images of corn, 4063 images of grape, 5507 images of orange, 2657 images of peach, 2475 images Bell paper, 2152 images of potato, 371 images of Raspberry, 5090 images of soybean, 1835 images of squash, 1565 images of strawberry and 18,162 images of tomato. In total, there are 54,309 image data from fungal, bacterial, mold, mite-diseased plants, and also healthy plants. As the data set is quite large, we have selected 19 classes among the 38 courses for training to minimize the training time.

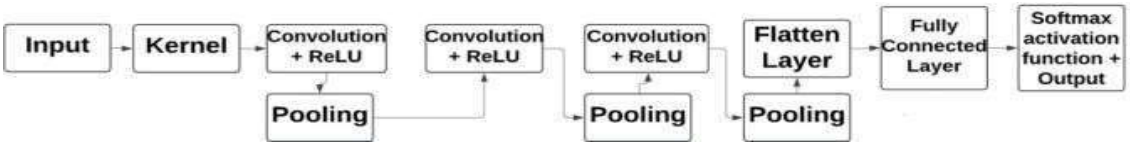


Fig. 1. Fundamental structure of CNN [1]

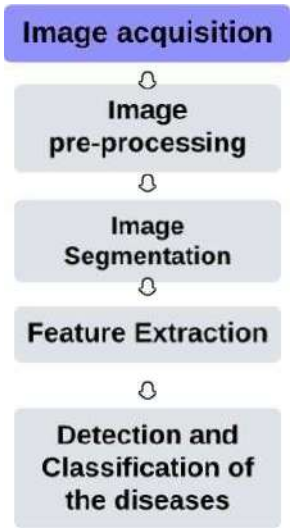


Fig. 1. Fundamental structure of CNN [1]

The methodology's flowchart showcased in this experiment is illustrated in Fig. 2. The first phase involved obtaining relevant images. Following this, the images underwent quality enhancement through pre-processing steps, which included noise reduction and contrast adjustment. Next, image segmen-tation was performed to break down the image into substantial areas such as healthy and diseased regions. Feature extraction then identified essential characteristics like color and texture from these divided regions. Finally, diseases

Volume 1 | Issue 1 | 2026
International Journal
of Multidisciplinary Science, Engineering & Technology. (IJMSET)

were detected and classified using machine learning or rule-based methods based on these extracted features. This experiment promi-nently utilized advanced CNN architectures such as AlexNet, GoogleNet, VGG19, ResNet50, and ResNet101.

AlexNet is a pre-trained model that adheres to the structure of the fundamental CNN model depicted in Fig. 1. It was proposed by Alex Krizhevsky and his team in 2012. [10]. For input, this model used an RGB image of shape 227*227*3. The first convolutional layer was then applied to have 96 filters of 11 * 11, a feature size of 55 * 55 * 96, a stride of 4, and a ReLU activation function. Next, the initial max-pooling was applied having a dimension of 3*3 and a stride of 2, the resulting feature map size was 27*27*96. After that, a subsequent convolutional operation was applied, this time filter size was 5*5 of 256 filters. This resulted in a feature map size of 27 * 27 * 256, and the ReLU activation function was applied to this layer. Again, the max pooling layer size of 3*3 was applied, stride 2, and the feature map size was 13*13*256. Next, the third convolutional operation was applied with 384 filter sizes of 3*3 and padding of 1. The resulting feature map size was 13*13*384. On top of that activation function, ReLU was applied. Again, the same convolutional layer was applied to the previous layer. Then, it had the last convolutional layer with 256 filters of dimensions 3*3, stride of 1, and padding of 1. The size of the feature map was 13*13*256. Then, a third max pooling layer was applied with dimensions of 3*3 and the resulting feature map of 6*6*256. After that, it had its earliest dropout layer, which was set to 0.5. Subsequently, it had its first fully connected layer with 4096 neurons with ReLU activation function. One more time, we had the second dropout layer with a rate of 0.5. It was followed by the second fully connected layer with 4096 neurons and the ReLU activation function. The final step involved a fully connected output layer containing 1000 neurons, utilizing the softmax activation function. This completes the description of AlexNet's architecture, which comprises a total of 62.3 million parameters.

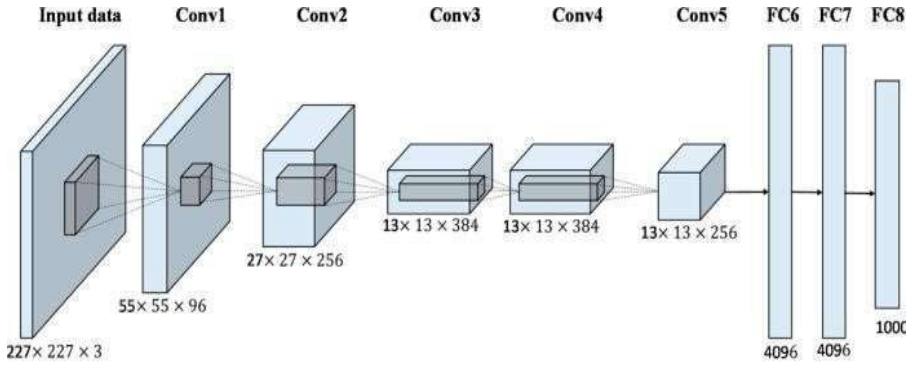


Fig. 4. Visual representation of AlexNet Architecture [11]

The VGG19 model is the same as the VGG16 model except for the fact that it has 19 convolutional layers. It consists of 16 convolutional layers. Also, it has three fully connected layers and one softmax layer. Dimensions of 224*224 of RGB photo

were used as input to this CNN architecture, meaning that the matrix dimension was 224*224*3. It used a kernel size of 3*3 without 1 pixel of stride, which enabled the model to cover the entire notation of the image. Special padding was used to preserve the special resolution of the picture. Max pooling was performed over a 2*2 pixel window with a stride dimension of 2. Then, the ReLU activation function was followed to introduce non-linearity to the architecture. The initial two layers had a dimension of 4096 and then a layer of 1000 channels in the three fully connected layers. Eventually, the last layer was the softmax function [12].

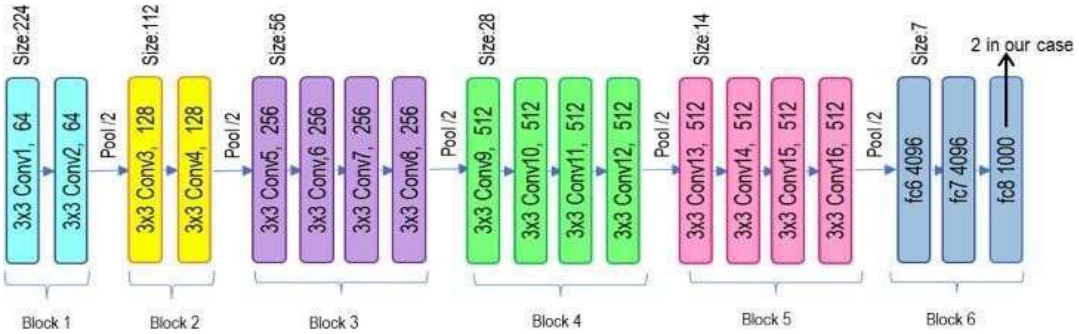


Fig. 5. Visual representation of VGG19 Architecture [13]

GoogLeNet was the winner of the 2014 ILSVRC, whereas AlexNet was the champion of 2012. [14] GoogleNet, also known as the Inception v1 network, introduced an inception layer. The network comprised 22 weight layers. However, the core building block of this network was the inception layer. It consisted of 3 * 3 convolutional layers, 5 * 5 convolutional layers, and 1 * 1 convolutional layer. All these were utilized together. The parameter number was 62 million. In the last stage, a crucial feature was introduced in this network, which was global pooling. This helped to reduce the total number of parameters. By performing a 1 * 1 convolutional network, the authors not only saved space but also increased the accuracy and training speed.

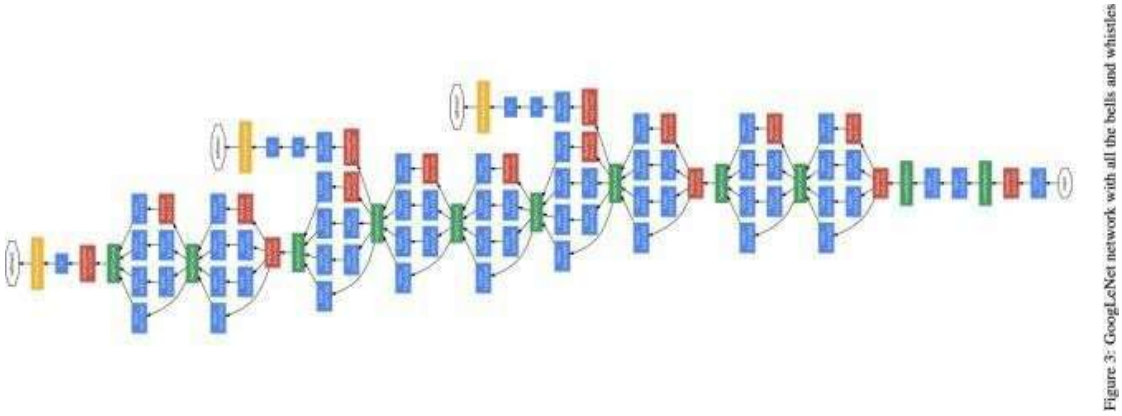


Figure 3: GoogLeNet network with all the bells and whistles

Fig. 6. Visual representation of GoogLeNet Architecture [14]

ResNet50 was introduced by Kaiming He et al. in 2015. The architecture uses a 224*224 image as input [15]. The next layer was zero padding. To create a feature map in the first stage, it had a convolutional layer. Later, this layer applied filters to the input. Then, batch normalization was applied for the normalization of the output of the previous layer. After that, an activation function ReLU was added to the application to introduce non-linearity. Max pooling was used after that to downsample the feature map to reduce its dimensions. Stage 1 to

Volume 1 | Issue 1 | 2026
International Journal
of Multidisciplinary Science, Engineering & Technology. (IJMSET)

5 refers to grouping the convolutional blocks and ID blocks. Each block might contain multiple blocks, and the number of blocks at each stage could vary. In the last stage, average pooling was applied to reduce the spatial dimensions one by one, essentially by taking the average of each feature. The flatting output from the pooling layer was then fed into the fully connected layer, which typically serves as a classifier. The fully connected layer mapped the features learned from the network to the desired number of output classes.

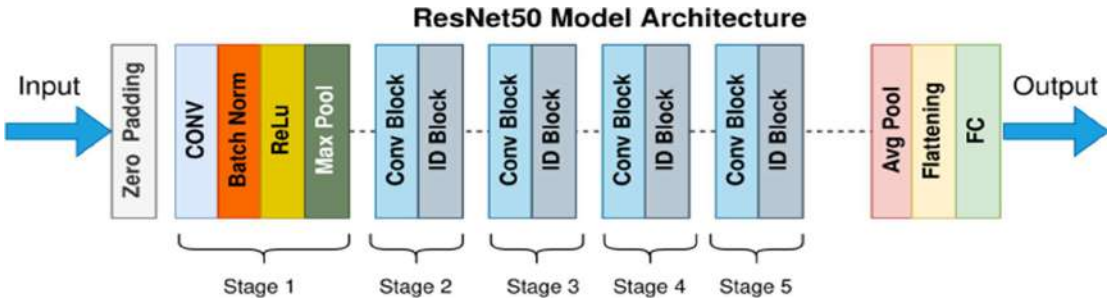


Fig. 7. Visual representation of ResNet50 Architecture [16]

In [17], the ResNet101 network was explained. The sole difference between the model and the CNN model ResNet50 is that the latter has 101 layers in depth. It further has five stages in total. There was just one convolutional layer in Stage 1. Three residual blocks, each with three layers, made up the second stage. Four residual blocks, each with three levels, made up the third level. There were twenty-three residual blocks in the fourth level, each with three levels. In the final level, there are three leftover blocks, each with three levels. When all the levels from the five stages were combined, there were 101 layers total. ResNet101 is usually used to get more complex features.

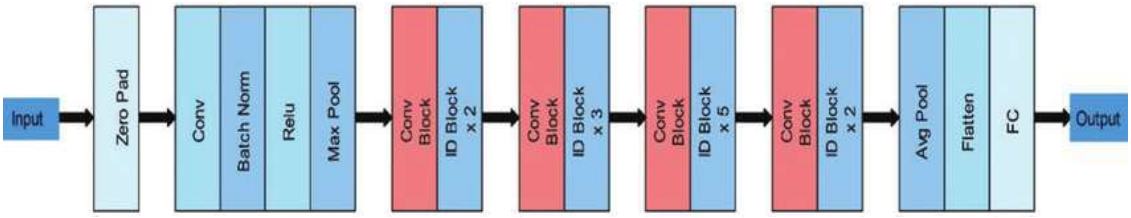


Fig. 8. Visual representation of ResNet101 Architecture [17]

4. RESULT ANALYSIS AND DISCUSSION

A. Accuracy and Loss

Fig. 9 shows the accuracy of the AlexNet model. The test accuracy of AlexNet was 77. 31%, and the loss was 65.44%. The precision of 77 % means that it can successfully detect 77% of the samples in the training and validation set. The loss of information indicates how well the model is performing and how much the predictions are the same as the actual data. A loss of 65% means that there is some room for improvement in this model. The test accuracy of GoogLeNet was 95.20%, and the final loss was 14.93%, which is a strong indicator that the model is performing better than AlexNet. This model correctly classifies 95 percent of the sample. Showing it's higher accuracy. A lower loss value suggests that the model predictions are closer to the actual labels. Lower loss with higher efficiency indicates GoogLeNet's excellent efficiency. The test accuracy of ResNet50 here was 76.96%, and the final loss was 54.44%, which is a close value to AlexNet. This

Volume 1 | Issue 1 | 2026
International Journal
of Multidisciplinary Science, Engineering & Technology. (IJMSET)

model correctly classifies almost 77% of the test dataset. For VGG19, the loss was 51.64%, and test accuracy was 92.81%. Although the correct classification performed by this model was around 93 %, the relatively high value of loss metric means that the model is performing well on accuracy, but it needs to improve the prediction confidence as well as data distribution more effectively. The final loss for ResNet101 was 10.29%, and the test precision was 97. 64%. The higher accuracy indicates this model has learned to generalize exceptionally well on new data. Both ResNet101 and GoogLeNet are highly effective. However, ResNet101 shows slightly higher performance than GoogLeNet in this case.

B. Training Time

The training time for AlexNet was 1 hour 46 minutes for 30 epochs. This model was trained for 40 and 50 epochs, but the training epoch was set to 30 due to overfitting. GoogLeNet took around 2 hours and 31 minutes for 8 epochs to complete. Early stopping was used here. It took 2 hours and 46 minutes to train the VGG19 model for optimum epoch 40. The training period of ResNet50 was 3 hours and 57 minutes for 50 epochs. ResNet101 took around 19.79 minutes or 1187.679s for the training execution. It stopped early at Epoch 8. It had early stopping parameters.

C. Evaluation Metrics

Precision is the measure of the accuracy of positive predictions(1).

$$Precision = (TP)/((TP) + (FP)) \quad (1)$$

where TP is True Positives, FP stands for False Positives, and FN stands for False Negative.

$$Recall = (TP)/((TP) + (FN)) \quad (2)$$

D. Model Complexity

From Table 1, if the parameters were chronologically or-dered. The first model that comes is GoogLeNet with 6.6 million parameters, ResNet50 with 25.6 million, ResNet101 with 44.5 million, AlexNet with 61 million, and VGG19 with 143.7 million. On the other hand, AlexNet has the lowest number of FLOPs (Floating Point Operations Per Second) with714 million, GoogLeNet with 1.5 billion, ResNet50 with 4.1 billion, ResNet101 with 7.9 billion and VGG19 19.6 billion. GoogLeNet has a comparatively smaller number of parameters and FLOPs with respect to other models.

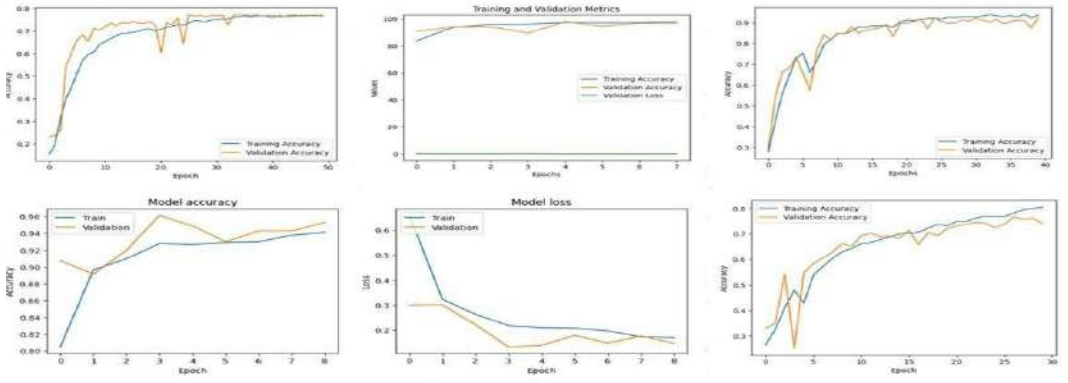


Fig. 9. ResNet50 model accuracy (top-left corner), ResNet101 model accuracy, VGG19 model accuracy, GoogLeNet model accuracy (bottom-left corner) and loss, and AlexNet model accuracy

Recall also referred to as the true positive rate, measures the effectiveness of the model in identifying all positive instances. (2).

ROC Curve is a graphical representation practiced to assess the binary classification of a model. Accuracy is evaluated only on the basis of a single threshold. However, the ROC curve calculates the performance of a classifier across all possible classification thresholds. In the ROC curve, the x-axis represents the False Positive Rate (FPR), and the y-axis represents the True Positive Rate (TPR). The diagonal dashed line represents the ROC curve of a random classifier, where FPR equals TPR.

AUC (Area Under Curve) values close to 1 represent the model's good performance. However, an AUC value of 0.5 means the model is random guessing. In the PR (Precision-Recall) curve, the x-axis represents recall, and the y-axis represents precision. Each line graph describes how precision and recall vary as the decision thresh-old for classifying instances is adjusted. The AP (average precision) value shows the entire performance of each class by averaging the precision across the recall levels. AP value close to 1 means higher performance of that class. A lower value indicates weaker performance.

Among the five models, the ROC and precision-recall performance of ResNet101 and GoogLeNet were higher. In Fig. 10, the AP values of GoogLeNet of all 19 classes were v ery close to 1. The lowest AP value here is 0.87. Again, the AUC values of all the classes for GoogLeNet were 1, which indicates excellent performance by this model-terms of ResNet101; almost all AP values were 1 or close to 1, except for classes 16 and 18. AP values were 0.72 and 0.78, respectively. In this case, GoogLeNet was slightly ahead of ResNet101. AUC values of all classes for ResNet101 were 1, 0.99 or 0.97. GoogLeNet also outperformed ResNet101 here. As the dataset is imbalanced, GoogLeNet showed good performance in identifying all the classes on test dataset.

5. CONCLUSION

In terms of accuracy, loss, and training time, among the five models ResNet101 performed exceptionally well, achieving the highest accuracy of 97 64% and completed the train-ing within 19 minutes, despite its higher model complexity resource requirements. Among the five models, considering accuracy, loss, and the time required for training, ResNet101 stood out with commendable performance, reaching an impres-sive accuracy of 97.64% and finishing the training process in just 19 minutes, albeit with increased resource requirements due to its higher model complexity. The performance of the other models was as follows: AlexNet (77%), GoogLeNet (95. 20%), ResNet50 (76. 96%) and VGG19 (92. 81%). Although ResNet101 excels in accuracy, its high complexity precision-recall deployment is in low-power environments. In contrast, GoogLeNet achieves an accuracy of 95.20%, providing balanced performance with notably fewer parame-ters and Floating-Point Operations Per Second, which gauges system computational power. This makes it well-suited for deployment on resource-limited devices such as the Raspberry Pi. Furthermore, GoogLeNet's performance in terms of ROC and precision-recall slightly surpasses that of ResNet101. Therefore, while ResNet101 is an excellent option for applica-tions prioritizing high accuracy without resource constraints, GoogLeNet is preferable for environments requiring efficient performance with minimal resource usage. Note that inference time was not considered in this study, and future research could include this important metric to better understand the practical deployment of these models.

6. REFERENCES

1. S. Forhad, K. Zakaria Tayef, M. Hasan, A. Shahebul Hasan, M. Za-hurul Islam, and M. Riazat Kabir Shuvo, "An autonomous agricultural robot for plant disease detection," in The Fourth Industrial Revolution and Beyond: Select Proceedings of IC4IR+, pp. 695–708, Springer, 2023.
2. H. Durmus¸, E. O. Gu¨nes¸, M. Kırcı, and B. B. U¨ stu¨ndag˘, "The design of general purpose autonomous agricultural mobile-robot:"agrobot"," in 2015 Fourth International Conference on Agro-Geoinformatics (Agro-geoinformatics), pp. 49–53, IEEE, 2015.
3. S. K. Tichkule and D. H. Gawali, "Plant diseases detection using image processing techniques," in 2016 Online International Conference on Green Engineering and Technologies (IC-GET), pp. 1–6, IEEE, 2016.
4. Xenakis, G. Papastergiou, V. C. Gerogiannis, and G. Stamoulis, "Applying a convolutional neural network in an iot robotic system for plant disease diagnosis," in 2020 11th International Conference on Information, Intelligence, Systems and Applications (IISA, pp. 1–8, IEEE, 2020.
5. S. Sladojevic, M. Arsenovic, A. Anderla, D. Culibrk, and D. Stefanovic, "Deep neural networks based recognition of plant diseases by leaf image classification," Computational intelligence and neuroscience, vol. 2016, 2016.
6. Thorat, S. Kumari, and N. D. Valakunde, "An iot based smart solution for leaf disease detection," in 2017 international conference on big data, IoT and data science (BID), pp. 193–198, IEEE, 2017.
7. K. R. Gavhale, U. Gawande, et al., "An overview of the research on plant leaves disease detection using image processing techniques," Iosr journal of computer engineering (iosr-jce), vol. 16, no. 1, pp. 10–16, 2014.
8. V. Udutalapally, S. P. Mohanty, V. Pallagani, and V. Khandelwal, "scrop: A novel device for sustainable automatic disease prediction, crop selection, and irrigation in internet-of-agro-things for smart agriculture," IEEE Sensors Journal, vol. 21, no. 16, pp. 17525–17538, 2020.
9. Hughes, M. Salathe´, et al., "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv preprint arXiv:1511.08060, 2015.
10. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," Communications of the ACM, vol. 60, no. 6, pp. 84–90, 2017.
11. X. Han, Y. Zhong, L. Cao, and L. Zhang, "Pre-trained alexnet archi-tecture with pyramid pooling and supervision for high spatial resolution remote sensing image scene classification," Remote Sensing, vol. 9, no. 8, p. 848, 2017.
12. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.
13. Khattar and S. Quadri, "Generalization of convolutional network to domain adaptation network for classification of disaster images on twitter," Multimedia Tools and Applications, vol. 81, no. 21, pp. 30437–30464, 2022.
14. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1–9, 2015.
15. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778, 2016.
16. K. C. Howlader, "The impact of pre-training models and fine-tuning on histopathology cancer images: An investigation through transfer learning-based analysis," Master's thesis, North Dakota State University, 2023.
17. Khan, M. A. Khan, M. Y. Javed, M. Alhaisoni, U. Tariq, S. Kadry, J.-Choi, and Y. Nam, "Human gait recognition using deep learning and improved ant colony optimization.," Computers, Materials & Continua, vol. 70, no. 2, 2022